CS 회고

Node.js DB 연동 · MVC API

1. MySQL2와 비동기 DB 연동

> Node.js의 `mysql2` 는 커넥션 객체를 통해 SQL을 실행하며, 조회 결과는 행 배열과 필드 정보로, 데이터 변경 결과는 영향 행 수 등의 메타데이터로 반환된다.

JDBC에서는 Connection, Statement, ResultSet이 데이터베이스 연동의 핵심 객체이다. SELECT는 결과 집합을 반환하고, INSERT, UPDATE, DELETE는 변경된 행 수를 반환한다. 트랜잭션이 필요한 경우 커밋과 롤백을 통해 작업 단위를 제어한다.

Node.js의 `mysql2` 도 데이터베이스 연결과 SQL 실행이라는 원리는 동일하다. 다만 JavaScript 환경에서는 결과를 배열과 객체 형태로 다루며, 일반적으로 `rows` 와 `fields` 를 구조 분해 할당하여 사용한다.

```
const [rows, fields] = await pool.query(
  'SELECT empno, ename FROM emp WHERE empno = ?',
  [7369]
);

console.log(rows[0].ename);
```

SELECT 결과의 `rows` 는 객체 배열이다. 따라서 첫 번째 행의 특정 컬럼은 `rows[0].컬럼명` 으로 접근한다. SQL에서 별칭을 지정하면 별칭 이름이 객체의 필드명으로 사용된다.

작업 유형	대표 SQL	주요 반환 값
조회	SELECT	행 배열과 필드 정보
등록	INSERT	영향 행 수와 생성 ID 정보
수정	UPDATE	영향 행 수와 변경 행 수
삭제	DELETE	영향 행 수

`query()` 와 `execute()` 는 모두 SQL 실행에 사용할 수 있다. 일반적으로 `execute()` 는 준비된 문장 방식으로 단일 SQL을 실행할 때 사용하며, 파라미터 바인딩을 함께 처리한다. `query()` 는 다중 SQL 실행을 지원할 수 있으나, 설정과 보안 영향을 고려해야 한다.

MySQL2는 Promise, 콜백, async/await 기반 구현을 지원한다. Promise 체이닝은 `then()` 과 `catch()` 로 결과와 오류를 연결하며, async/await는 비동기 작업을 위에서 아래로 읽히는 코드 형태로 작성할 수 있게 한다.

- **Promise** 방식은 `then()` 과 `catch()` 를 체이닝하여 성공과 실패 흐름을 처리한다.
- **콜백** 방식은 실행 결과를 함수의 매개변수로 전달받아 처리한다.
- **async/await** 방식은 Promise 완료를 기다리는 코드를 직관적으로 작성한다.
- 비동기 데이터베이스 작업은 오류 처리를 위해 `try/catch` 와 함께 작성하는 것이 일반적이다.

2. 커넥션 풀과 자원 관리

> 커넥션 풀은 데이터베이스 연결을 미리 생성하여 재사용하며, 요청마다 연결을 새로 만드는 비용을 줄이고 동시 접속을 관리한다.

단일 연결은 `createConnection()` 으로 만들 수 있으며, 작업이 끝나면 연결을 종료해야 한다. 반면 커넥션 풀은 `createPool()` 로 생성하고, 필요할 때 연결을 대여한 뒤 반환하는 구조이다.

커넥션 풀은 동시에 사용할 수 있는 연결 수를 제한하고, 모든 연결이 사용 중일 때 새 요청을 대기열에 넣을 수 있다. 따라서 서버가 여러 요청을 동시에 처리하는 환경에서 단일 연결보다 안정적인 운영 방식이 된다.

```
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: '127.0.0.1',
  user: 'root',
  password: '비밀번호',
  database: 'test',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
  dateStrings: true
});
```

`connectionLimit`은 동시에 사용할 최대 연결 수이다. 이는 전체 사용자 수가 아니라 같은 시점에 데이터베이스 작업을 수행할 수 있는 연결의 상한이다. `waitForConnections`가 `true`이면 연결이 모두 사용 중일 때 후속 요청은 대기한다.

설정 항목	의미	활용 관점
`connectionLimit`	풀에서 유지할 최대 연결 수	동시 DB 작업량을 제어한다.
`waitForConnections`	연결 부족 시 대기 여부	일시적 트래픽 증가를 처리한다.
`queueLimit`	대기열 최대 크기	0은 무제한 대기열을 의미한다.
`dateStrings`	날짜 값을 문자열로 반환	날짜 직렬화 형식을 단순화한다.

풀 자체로 실행한 쿼리는 풀에서 연결을 관리한다. 그러나 `pool.getConnection()`으로 개별 연결을 빌렸다면 작업 후 `release()`로 반환해야 한다. 단일 연결은 일반적으로 `end()`로 종료한다.

- `createConnection()`으로 생성한 단일 연결은 `end()`로 종료한다.
- `createPool()`에서 대여한 개별 연결은 `release()`로 풀에 반환한다.
- 풀 전체를 종료해야 하는 경우에는 `pool.end()`를 사용한다.
- 트랜잭션 중 오류가 발생하면 롤백 후 자원을 정리해야 한다.

3. Express REST CRUD API

> REST API는 HTTP 메서드와 URL로 리소스 조작 의도를 표현하며, 조회 결과와 처리 상태는 JSON 및 HTTP 상태 코드로 응답한다.

사원 리소스를 예로 들면 전체 조회는 `GET /emps`, 단건 조회는 `GET /emps/:empno`처럼 구성할 수 있다. URL 경로의 `:empno`는 동적 파라미터이며, Express에서는 `req.params.empno`로 값을 추출한다.

```
app.get('/emps/:empno', async (req, res) => {
  const [rows] = await pool.execute(
    'SELECT * FROM emp WHERE empno = ?',
    [req.params.empno]
  );

  if (rows.length === 0) {
    return res.status(404).json({ message: '사원이 없습니다.' });
  }

  res.json(rows[0]);
});
```

등록과 수정에서는 클라이언트가 전송한 JSON 본문을 읽어야 한다. 이를 위해 `express.json()` 미들웨어를 등록해야 하며, 파싱된 데이터

는 `req.body`에서 확인한다.

4. SQL 바인딩과 입력 검증

> 사용자 입력값은 SQL 문자열에 직접 연결하지 않고 플레이스홀더와 파라미터 배열로 전달해야 하며, 서버는 데이터베이스 처리 전에 유효성 검사를 수행해야 한다.

SQL 문자열 결합은 입력값이 SQL 문법으로 해석될 위험이 있다. 특히 검색 조건, 등록 데이터, 수정 데이터에 사용자 입력을 직접 이어 붙이면 **SQL 인젝션**으로 데이터가 노출·변조·삭제될 수 있다.

```
const sql = 'SELECT * FROM emp WHERE ename LIKE ?';
const params = [`%${ename}%`];

const [rows] = await pool.execute(sql, params);
```

물음표 플레이스홀더는 SQL 구조와 데이터 값을 분리한다. 값은 두 번째 인수의 배열로 전달하며, 라이브러리가 안전한 바인딩 과정을 처리한다. LIKE 검색에서는 와일드카드 `%`까지 값에 포함하여 전달할 수 있다.

서버 검증은 프론트엔드 검증과 목적이 다르다. 프론트엔드는 사용자 경험을 위해 빈 값, 길이, 형식 등을 빠르게 확인한다. 서버는 외부 요청을 신뢰하지 않고 비즈니스 규칙, 데이터 타입, 중복 여부, DB 제약 조건을 최종 검증한다.

```
function validateEmp(emp) {
  const errors = [];

  if (!emp.empno || Number.isNaN(Number(emp.empno))) {
    errors.push('사원 번호는 숫자 필수값입니다.');
  }

  if (!emp.ename) errors.push('사원명은 필수값입니다.');
  if (!emp.job) errors.push('직무는 필수값입니다.');

  return errors;
}
```

검증 오류는 오류 메시지 배열로 관리할 수 있다. 배열 길이가 0보다 크면 SQL을 실행하지 않고 **400 Bad Request**로 즉시 응답한다. 기본키 중복처럼 DB에서 확인되는 충돌은 예외 코드에 따라 **409 Conflict**로 구분한다.

5. 검색과 페이징 처리

> 검색 조건은 쿼리 스트링에서 추출하여 동적으로 SQL에 추가하고, 페이징은 `LIMIT` 과 `OFFSET` 을 이용해 특정 범위의 결과를 반환한다.

검색 API는 `req.query`에서 쿼리 스트링 값을 가져온다. 예를 들어 `/emps/search?ename=김&job=MANAGER` 요청에서 `ename`과 `job`은 `req.query`로 추출한다. 조건이 존재할 때만 SQL과 파라미터 배열에 조건을 추가한다.

```
let sql = 'SELECT * FROM emp WHERE 1 = 1';
const params = [];

if (ename) {
  sql += ' AND ename LIKE ?';
  params.push(`%${ename}%`);
}

if (job) {
  sql += ' AND job = ?';
  params.push(job);
}
```

페이징에서는 페이지 번호와 페이지당 행 수를 숫자로 변환하고, 잘못된 값에는 기본값을 부여한다. `OFFSET`은 앞에서 건너뛸 행 수이며 `(page - 1) * size` 공식으로 계산한다.

```
const page = Math.max(parseInt(req.query.page) || 1, 1);
const size = Math.max(parseInt(req.query.size) || 10, 1);
const offset = (page - 1) * size;

const [rows] = await pool.execute(
  'SELECT * FROM emp LIMIT ?, ?',
  [offset, size]
);
```

라우터 선언 순서도 중요하다. `/emps/search` 같은 고정 경로는 `/emps/:empno` 같은 동적 경로보다 먼저 선언해야 한다. 그렇지 않으면 `search`가 사원 번호 파라미터로 해석되어 경로 충돌이 발생할 수 있다.

6. MVC 구조와 Sequelize ORM

계층	주요 책임	예시
Config	DB 연결과 환경 설정을 관리한다.	`database.js`
Model	테이블 구조와 DB 접근 대상을 정의한다.	`user.model.js`
Service	비즈니스 로직과 데이터 처리 흐름을 담당한다.	`user.service.js`
Controller	요청 데이터를 받고 HTTP 응답을 생성한다.	`user.controller.js`
Router	HTTP 메서드와 URL을 컨트롤러에 연결한다.	`user.route.js`

Express 프로젝트는 기능을 한 파일에 모두 작성할 수도 있지만, 규모가 커질수록 역할별 분리가 필요하다. 일반적으로 설정, 모델, 서비스, 컨트롤러, 라우터를 분리하면 코드의 책임과 변경 범위를 명확히 관리할 수 있다.

Sequelize

Sequelize는 Node.js에서 관계형 데이터베이스를 객체 중심으로 다룰 수 있게 하는 **ORM**이다. 모델을 정의하면 `findAll()`, `findByPk()`, `create()`, `update()`, `destroy()` 같은 메서드로 SQL 작성량을 줄일 수 있다.

```
const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  name: {
    type: DataTypes.STRING,
    allowNull: false
  },
  addr: {
    type: DataTypes.STRING
  }
}, {
  tableName: 'user',
  timestamps: false
});
```